TRANSACTION Processing Concept

- A transaction is a set of instruction which perform a logical unit of work and that must be atomic in nature.
- Transaction in database must be atomic

Transaction	Process
	I_1
	I_2
	I_3
	⋮
	I_{100}

Transaction

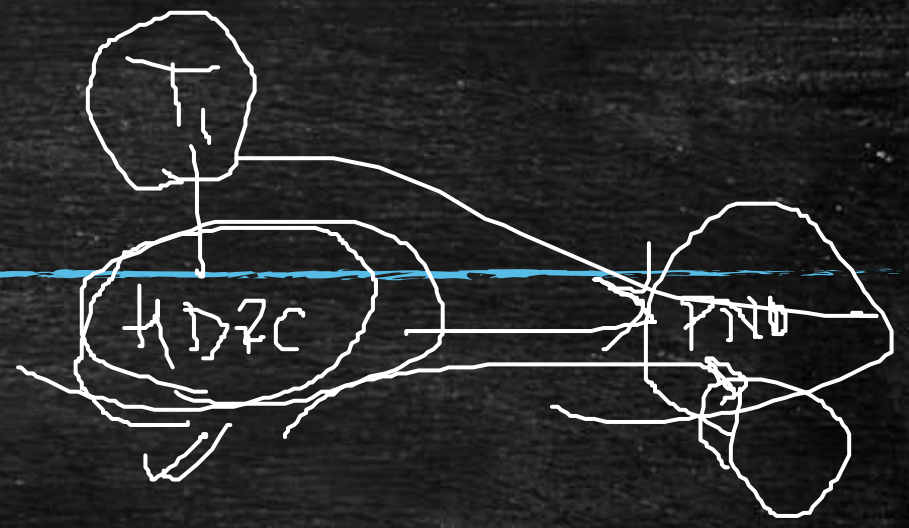
R(A)
A = A - 10
W(A)
R(B)
B = B + 10
W(B)

$A = \cancel{100}$ 90

$B = \cancel{20}$ 30

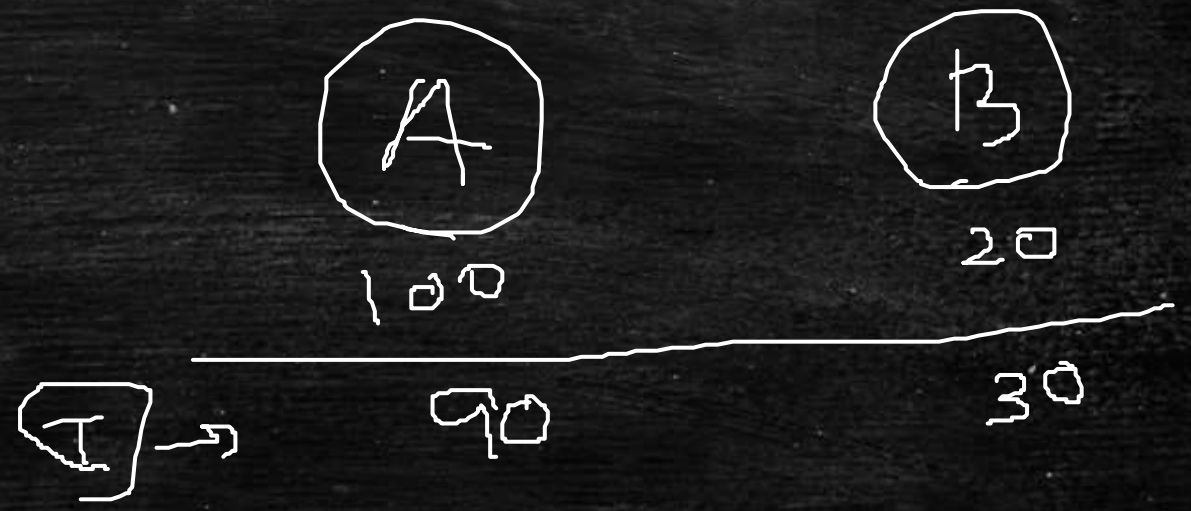
Failure

ACID Properties in dbms

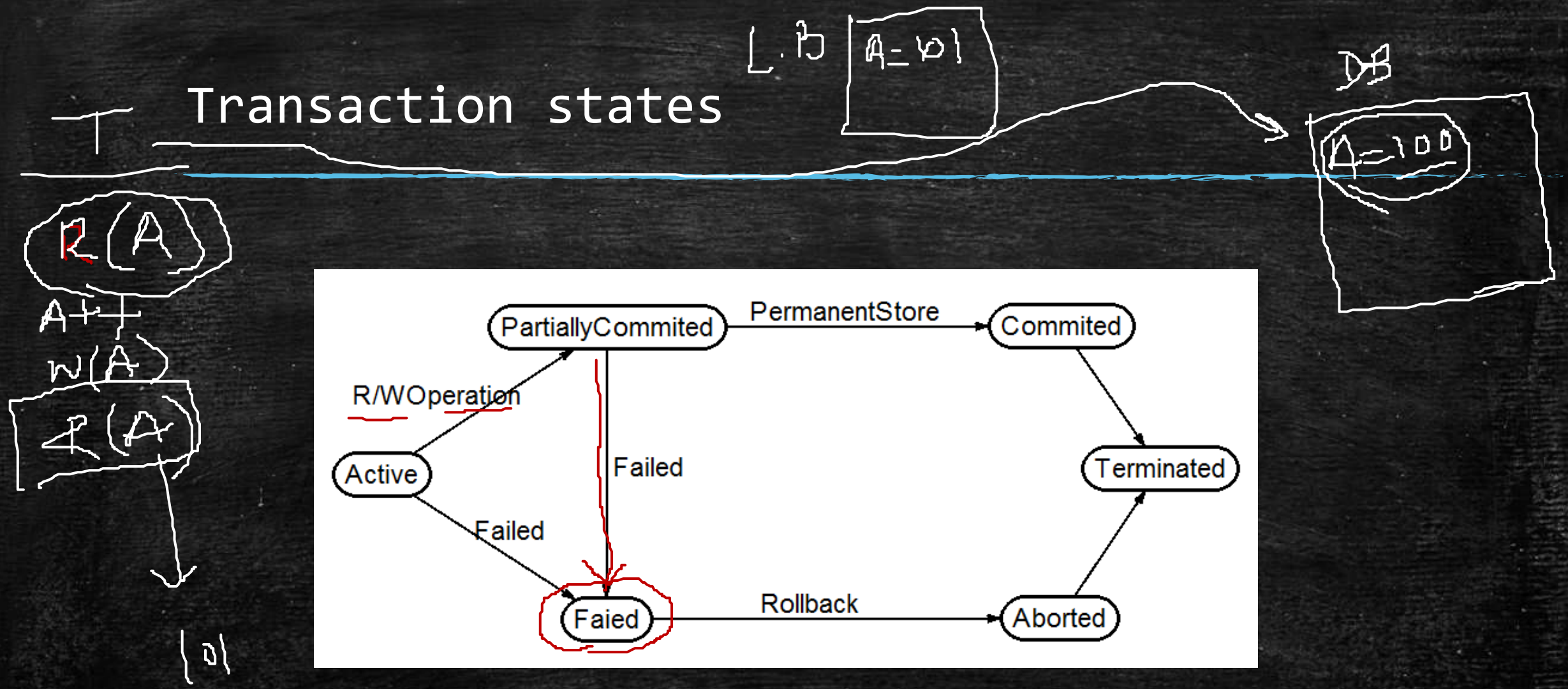


Transaction ACID Properties

1. Atomicity – (either all instruction or None)
2. Consistency – (A database must be consistent before and after the transaction)
3. Isolation(Logical Isolation)
4. Durability

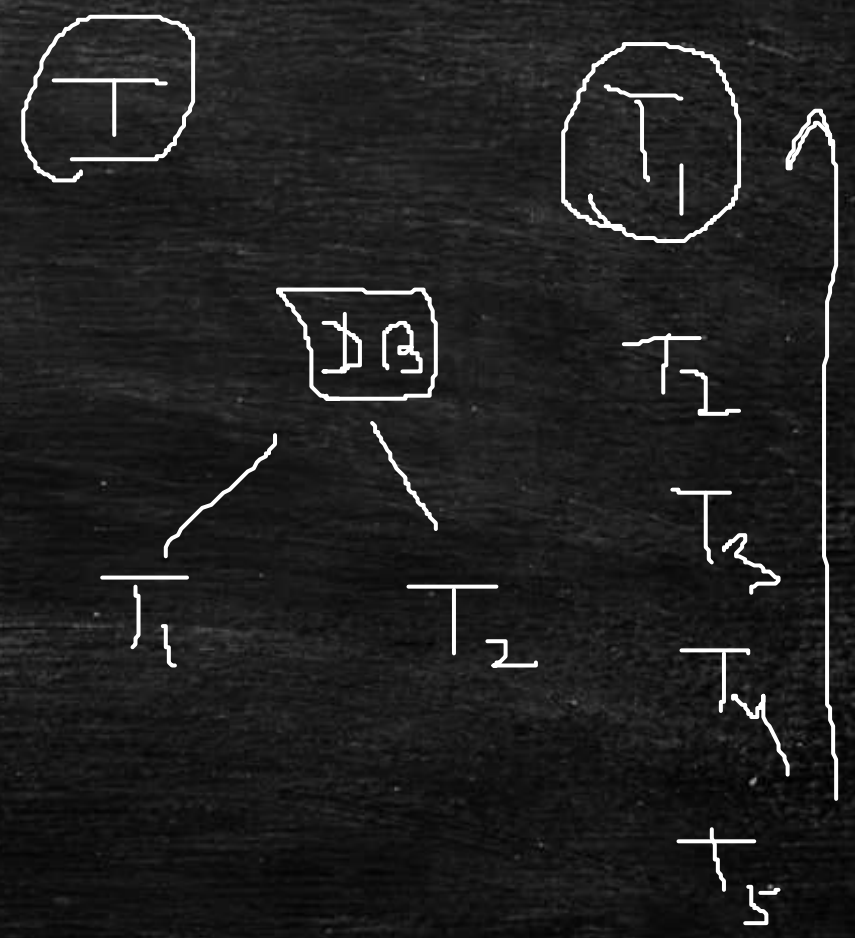


Transaction states



Advantage of concurrency

1. Waiting Time ↓
2. Response Time ↓
3. Resource Utilization ↑
4. Efficiency ↑



Schedule

T1	T2
R(A)	R(B)
W(A)	W(B)
R(B)	R(A)
W(B)	W(A)

T1	T2
	R(B)
	W(B)
	R(A)
	W(A)
R(A)	
W(A)	
R(B)	
W(B)	

T1	T2
R(A)	
W(A)	
	R(B)
	W(B)
R(B)	
	R(A)
W(B)	
	W(A)

- If we have n1 instruction in Transaction T1 and n2 instruction in Transaction T2 then after schedule of T1 and T2 we need to have total n1+ n2 Instruction
  - Serial Schedule: When we start a second transaction after completion of first that is serial schedule
  - Non-Serial Schedule: When context switching happen in between two transaction i.e Non- Serial

Dirty Read Problem

DB = A = 10

11

- Dirty Read- if a transaction reads a value of a uncommitted transaction instead of main database value then that is a Dirty Read

Local Buffer

A = 11

A++

T1	T2
1 R(A)	
2 W(A)	
	3 R(A)
	4 Committed
5 Committed	

11

Unrepeatable read problem

- If we didn't get same value after multiple read operation then scenario is known as unrepeatable read.
- Violation of isolation of Transaction

T1	T2
R(x)	
	R(x)
W(x)	
	R(x)

Phantom Read Problem

x = 10

T1	T2
R(x)	
	R(x) → 10
Delete(x)	
	R(x)

Lost update problem (write write conflict)

DB | A=5

T1	T2
R(A)	
W(A)	
	W(A)
	Commit
Commit	

A=10

A++

A=11

L.B

Blind write

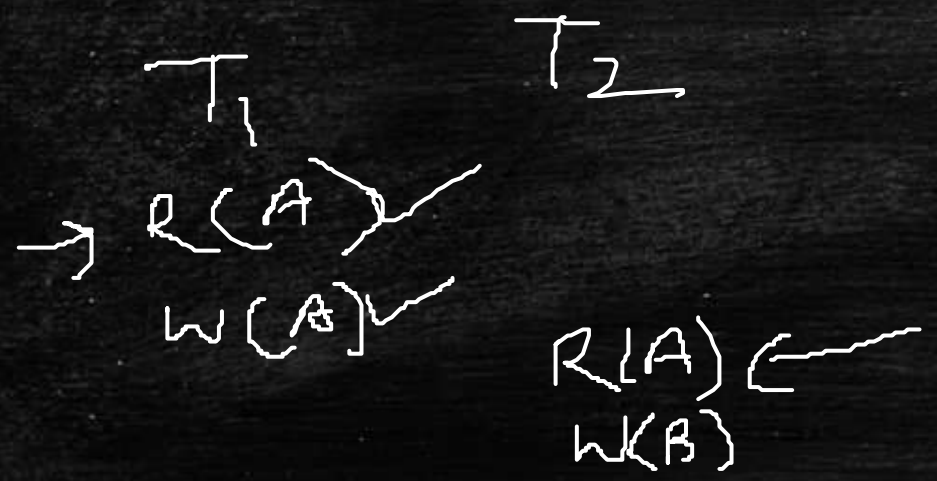
=50

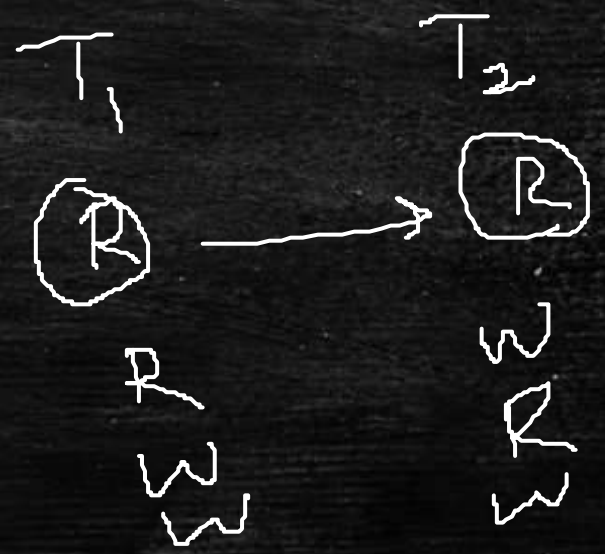
W(A)

Conflict serializability

R W

- To maintain consistency we have to check whether non serial transaction can be converted into serial transaction
- For that we need to do swapping process.(Swapping of Instruction)
- NOTE:- two instructions are said to be conflicting when they belong to different transactions but they must operate on same data value and one of them must be a write operation.





Conditions for conflict

1. Different Transaction
2. Same data value
3. One of operation must be write instruction

T_1	T_2
R(A)	R(A)

T_1	T_2
W(A)	W(B)

NOTE: if conflict occurs then we cannot swap the instructions

Conflict Serializabilty :- if we convert a non serial transaction into serial transaction by swapping of non-conflicting instruction then this schedule is known to be conflict serializabilty

NOTE: if a schedule is conflict serializable then it if going to be consistent

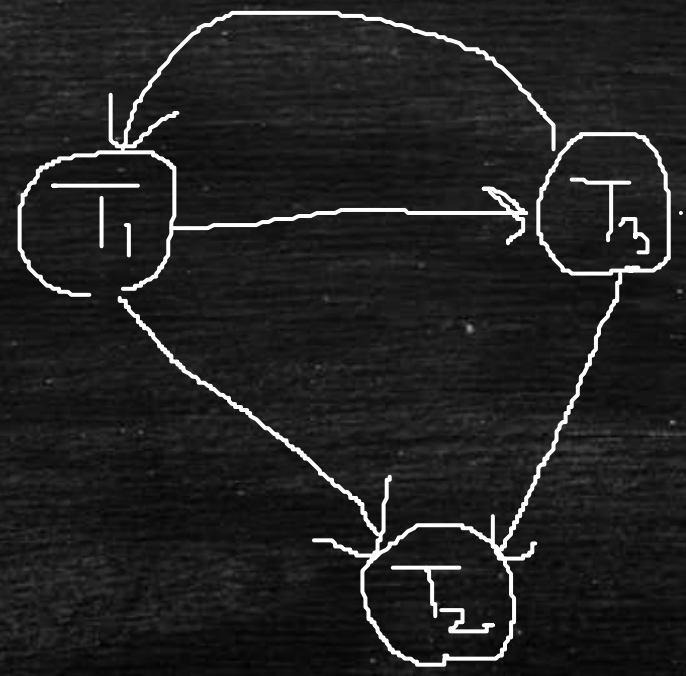
- Find schedule is conflict serializable or not ?

R(X) → W(X)

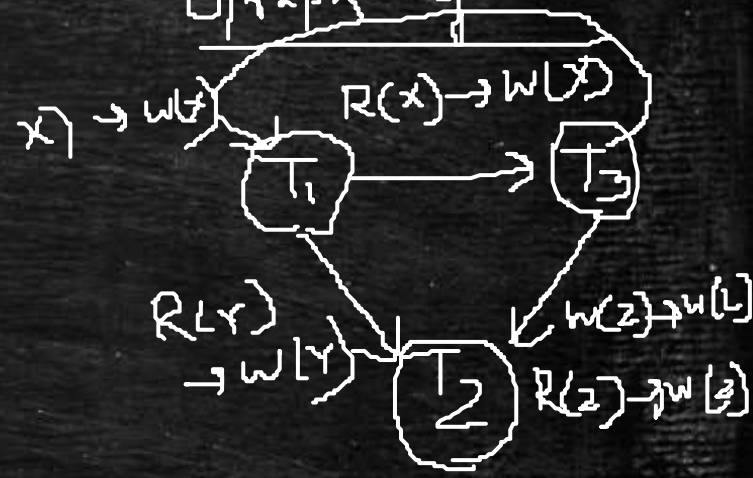
$T_1 \rightarrow T_3$

T1	T2	T3
R(X)		
		R(Z)
		W(Z)
	R(Y)	
R(Y)		
	W(Y)	
		W(X)
	W(Z)	
W(X)		

Not Conflict Serializable



Graph of Transaction



T_1

T_3

T_2

- Find is schedule is conflict serializable or not?

T1	T2	T3
R(X)		
	R(Y)	
		R(Y)
	W(Y)	
W(X)		
		W(X)
	R(X)	
	W(X)	

- Find is schedule is conflict serializable or not?

T1	T2	T3	T4
	R(X)		
		W(X)	
		commit	
W(X)			
commit			
	W(Y)		
	R(Z)		
	commit		
			R(X)
			R(Y)
			commit

View Serializability

- A schedule is view serializable if it is view equivalent to a serial schedule
- If a schedule is conflict serializable, then it will be view serializable.
- The view serializable which does not conflict serializable contains blind writes.
- NOTE: if a schedule is view serializable but not conflict serializable then there must exist a BLIND WRITE

Conflict Serializability

- To check the view serializablity we have to compare the non serial and various serial schedule and if one of the serial schedule is view equivalent to non serial schedule then that schedule will be view serializable
- To prove equivalence of two schedule we need to check three conditions
  - Initial Read
  - Final Write
  - Intermediate Reads

- Find if two schedule are view equivalent or not ?

S	
T1	T2
R(a)	
w(a)	
	R(a)
	w(a)
R(b)	
w(b)	
	R(b)
	w(b)

S`	
T1	T2
R(a)	
w(a)	
R(b)	
w(b)	
	R(a)
	w(a)
	R(b)
	w(b)

- Find if 'S' is view serializable or not

S		
T1	T2	T3
R(a)		
	w(a)	
w(a)		
		w(a)

Recoverability

- Recoverable schedule – Even if a schedule is consistent or conflict serializable then also if any error or failure occurs in between the transaction then it may lead to inconsistency or irrecoverable state. This type of schedule is known as irrecoverable schedule
- The above condition is possible only when some transaction depends on other transaction and there exist a case of Dirty Read

T1	T2
R(A)	
A=A+10	
W(A)	
	R(A)
	A=A-5
	W(A)
	Commit
Commit	

- In both failure there will be no irrecoverable state
- This is because T2 is having a Dirty Read but it commits after T1
- So schedule is recoverable

T1	T2
R(A)	
A=A+10	
W(A)	
	R(A)
	A=A-5
	W(A)
Commit	
	Commit

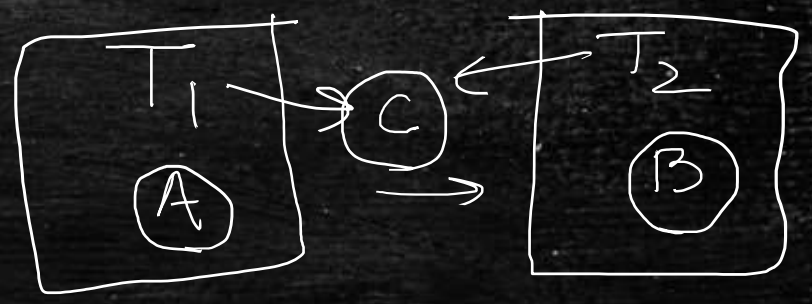
Cascadeless schedule

- The schedule 'S' is conflict serializable because it is serial
- We have dirty read in T2 and T3
- Dirty Read Order is T1→T2→T3
- If commitment will be also in same order then the schedule will be recoverable. But in above case it is not , T2→T1→T3
- So above schedule is irrecoverable
- If commitment is in same order of dirty read then consistency is maintained and rollback is possible
- This scenario is known as cascading rollback
- A schedule is said to be Cascadeless if there is no cascading rollback in that schedule

S		
T1	T2	T3
R(A)		
W(A)		
	R(A)	
	W(A)	
		R(A)
	Commit	
Commit		
		Commit

Concurrency control technique

- In this we are going to study protocols that guarantees to generate schedule which satisfy the properties which leads to concurrency like conflict serializability
- For conflict serializability we must avoid conflicting instructions.
- We must remember the three conditions of conflicting instruction.
- So the actual problem is that different transaction trying to access data at same time



- To control concurrency we need to have sharp approach for conflict serializability
  - Time Stamping Protocol
  - Locked Based Protocol
    - 2-Phase Locking
    - Graph Based Protocol
  - Validation Protocol

Time stamping ordering protocol

- Basic idea of Time Stamping is to decide the order between the transactions before that enters into the system so that in case of conflict during execution we can resolve the conflict using ordering.
- The reason we call time-stamping is because for stamping we use value of system clock

T_1 13:55

T_2 13:56

$TS(T_1) < TS(T_2)$

Two ideas of Time Stamping

1. Time stamping with Transaction – with each transaction T_i, we associate a time stamp donated by $T.S(T_i)$
   - It is the value of the system clock when a transaction enters into the system so if a new transaction T_j enters after T_i then $T.S(T_i) < T.S(T_j)$ always unique, will remain fixed through the execution
   - Also determine serializability order of $T.S(T_i) < T.S(T_j)$ then system ensure that in the resultant conflict serializable schedule. T_i will execute before T_j

2. Time Stamping with data items – for each data item Q, protocol maintains two time stamps
   - W-time stamp(Q) – is the largest time stamp of any transaction that executed write(Q) successfully
   - R-time stamp(Q)- is the largest timestamp of any transaction that executed read(Q) successfully

- T_i request for Read(Q)
  - If $T.S(T_i) < W.T.S(Q)$ means T_i needs to read a value of Q that was already overwritten. Hence request must be rejected and T_i must rollback
  - If $T.S(T_i) > W.T.S(Q)$ means operation can be allowed and R.T.S(Q) will be max(R.T.S(Q), $T.S(T_i)$)
- T_i request for write (Q)
  - If $T.S(T_i) < R.T.S(Q)$ means value of Q that T_i is producing was needed previously and the system assumed that the value would never be produced hence rejected and rollback
  - If $T.S(T_i) < W.T.S(Q)$, here T_i is attempting to write an obsolete value of Q so Rejected and Rollback

$T.S(T_i) < W.T.S(Q)$

10:00 11:00

$T.S(T_i) > R.T.S(Q)$

T_i w(Q) 10:00 9:00

$T.S(T_i) > W.T.S(Q)$

w(Q)

$T.S(T_i) = 10:00$ $T.S(T_x) = 09:00$

R(Q)

Properties of Time stamping protocol

$T_1 = 9:00$

$T_2 = 10:00$

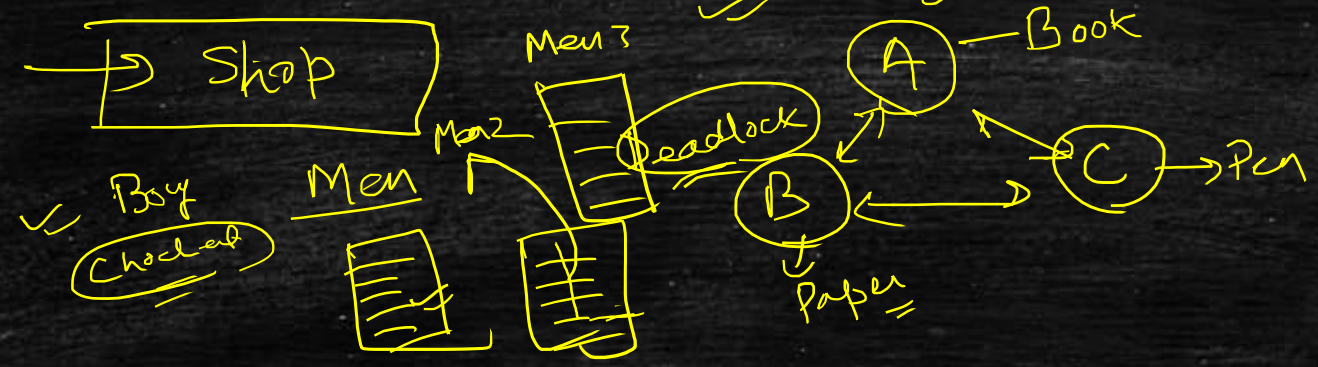
$T_3 = 11:00$

- It ensure conflict serializability
- Possibility of dirty read, commit, irrecoverable schedule and cascading rollback are possible
- Here either we allow ~~of~~ or reject so no idea of deadlock
- It may suffer from starvation and it is relatively slow

Deadlock

Assignment

1. Book
2. Pen
3. Paper



Lock based protocol

- To achieve the consistency, isolation is the most important idea.
- Locking is simplest idea to achieve isolation i.e first obtain a lock on a data item then perform a desired operation and then unlock it.
- To provide better concurrency along with isolation we use different modes of locks:
  1. Shared mode : denoted by Lock-S(Q) – this transaction can perform read operation. Any other transaction can obtain same lock on same data at same time.
  2. Excusive mode : denoted by Lock-X(Q) – this transaction can perform both read / write operation, any other transaction can not obtain either shared / exclusive mode lock.

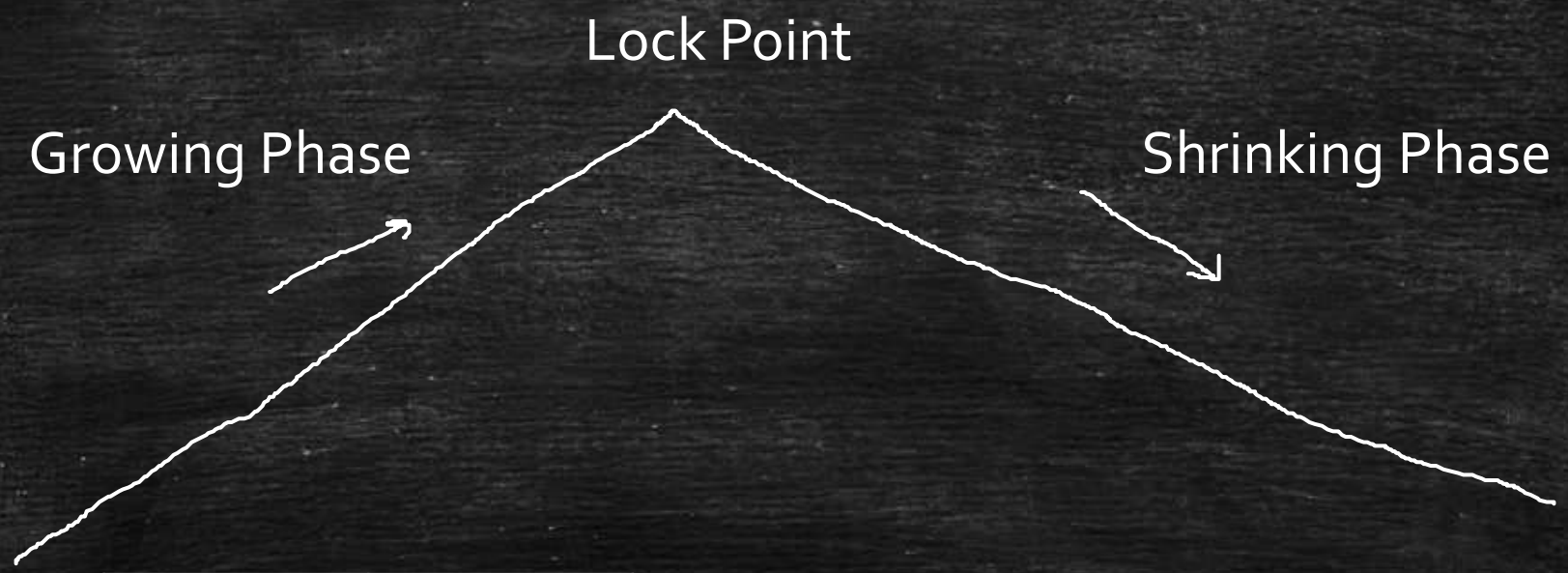
Lock based protocol

- Here if we do unlocking inconsistency will arise and if we do not unlock then concurrency will be poor
- We require transaction to follow some set of rules for locking and unlocking of data item. Like 2PL or graph based protocol

T1	T2
Lock-X(A)	
R(A)	
W(A)	
Unlock-(A)	
	Lock S(B)
	R(B)
	Unlock (B)
Lock X(B)	
R(B)	
W(B)	
Unlock(B)	
	Lock S (A)
	R(A)
	Unlock (A)

Two- Phase locking (2-PL) / Basic 2PL

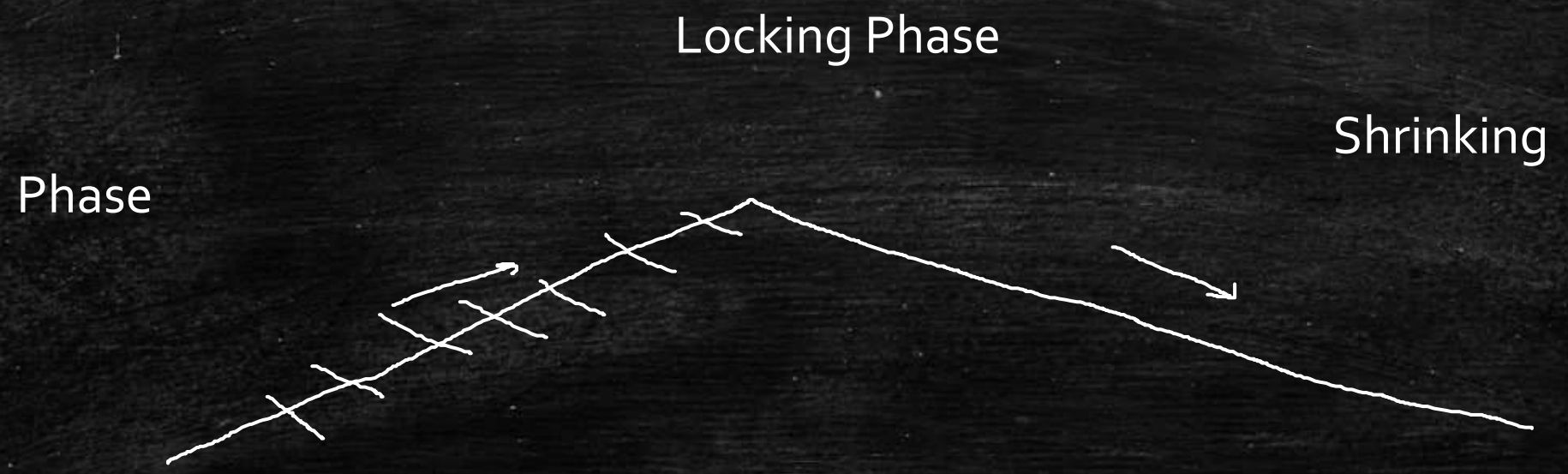
- This protocol requires that each transaction in a schedule will be two phased
  - Growing phase
  - Shrinking phase
- In growing phase, transaction can only obtain locks but cannot release any lock
- In shrinking phase, transaction can only release locks but cannot obtain any lock
- Transaction can perform read/write operation both in growing/ shrinking phase
- Ensures conflict serializability, the order of serializability is order in which transaction reaches lock point.
- Do not ensure freedom of deadlock



T1	T2
Lock-S(A)	
R(A)	
	Lock X(B)
	R(B)
	W(B)
Lock (B)	
	Lock(A)

Conservative / Static 2PL

- There is no growing phase transaction, first will acquire all the locks required and then directly start from lock point
- It all locks are not available then transaction must release the lock acquired so far and wait



- Shrinking Phase works as usual and transaction can unlock any data item
- Here we must have all the knowledge that what data items will be required during execution
- Conflict serializability independent from deadlock
- Possibility of irrecoverable schedule and cascading rollbacks

Rigorous 2PL

- It is an improvement over 2PL protocol where we try to ensure recoverability and cascadlessness.
- Rigorous 2PL requires that all the locks must be held until transaction commits i.e there is no shrinking phase in the life of all transaction
- Will ensure conflict serilizability, recoverability, cascadlessness
- Suffer from deadlock and inefficiency

Strict 2PL

- It is an improvement over rigorous 2PL
- In the shrinking phase Unlocking of exclusive locks are not allowed but unlocking of shared lock can be done
- All the properties are same as that of rigorous 2PL but it provide better concurrency
- C.S, concurrency, cascedlessness, recoverability are ensured
- Deadlock problem will remain as it is

Graph Based protocol

- If we wish to develop locked based protocol that are not based on 2PL, we need additional information that how each transaction will access the data
- There are various model that can give addition information each differing in the amount of info provided
- One idea is to have knowledge about the order in which the database items will be accessed
- We impore partial ordering → on set all data items D= { d1,d2,d3…..dn} if di→dj then any transaction accessing both di and dj must access di before dj
- P.O (Partial Ordering) may be because of logical or physical organization or only because of concurrency control
- After P.O set of all data items 'D' will now be viewed as directed acyclic graph called database graph
- For sake of simplicity we follow two restrictions
  - Will study graph that are rooted tree
  - Will use only exclusive mode locks

Tree Protocol

- Each Transaction Ti can lock on data item Q with following rules
  - First Lock by Ti may be on any data item
  - Subsequently a data item Q can be locked by Ti only if the parent of Q is currently locked by Ti
  - Data item may be unlocked at any time
  - Data item Q that has been locked and unlocked by Ti can not subsequently be relocked by Ti

Properties of Tree Protocol

- All schedule that are legal under tree protocol are conflict serializable
- Tree protocol ensures freedom from deadlock
- Tree protocol do not ensure recoverability and Cascadelessness
- Easy unlocking is possible which leads to shortest waiting time and increase concurrency
- A transaction may have to lock data items that it does not access lead to overhead, waiting time and decrease in concurrency
- Transaction must know exactly what data item are to be accessed
- Recoverability and Cascadelessness can be provided by no unlocking before commit

Validation Based Protocol

Validation phase is also known as optimistic concurrency control technique. In the validation based protocol, the transaction is executed in the following three phases:

1. **Read phase:** In this phase, the transaction T is read and executed. It is used to read the value of various data items and stores them in temporary local variables. It can perform all the write operations on temporary variables without an update to the actual database.

2. **Validation phase:** In this phase, the temporary variable value will be validated against the actual data to see if it violates the serializability.

3. **Write phase:** If the validation of the transaction is validated, then the temporary results are written to the database or system otherwise the transaction is rolled back.

Validation Based Protocol

Here each phase has the following different timestamps

- **Start(Ti):** It contains the time when Ti started its execution.
- **Validation (T_i):** It contains the time when Ti finishes its read phase and starts its validation phase
- **Finish(Ti):** It contains the time when Ti finishes its write phase.
- This method is not favourable for longer transactions because they are more likely to have conflicts and might be repeatedly rolled back due to conflicts with short transactions.

Validation Based Protocol

- Advantages :
  - **Avoid Cascading-rollbacks**
  - **Avoid deadlock**
- **Disadvantage:**
  - **Starvation**